END-TO-END DATA ANALYTICS CASE STUDY

Supply Chain & Logistics | Data Cleaning, KPI Analysis and Financial Impact

A practical, quantitative workflow using Python, SQL and Power BI

CASE PARAMETER	ILLUSTRATIVE ASSUMPTION
Analysis period	Six months: January–June
Order volume	50,000 orders
Logistics cost	S/ 2,400,000
Target OTIF	95%
Business issue	Rising transportation costs and late or incomplete deliveries

Important: All figures and sample outcomes in this document are hypothetical. They are designed to demonstrate an end-to-end analytical method and do not represent actual company results.

1. Business Problem and Objectives

A distribution company has seen transportation costs rise while delivery performance has deteriorated. Management requests an evidence-based diagnosis, an estimate of the financial opportunity, and a practical plan to improve service levels.

The analysis must answer the following questions:

- What proportion of source records contains duplicates, missing values, invalid dates, or inconsistent quantities?
- What is the actual On Time In Full (OTIF) rate, and how does it change over time?
- Which distribution centers, carriers, routes, or products are associated with the highest failure rates?
- What is the cost per order, and which inefficiencies create measurable additional expense?
- Which operational improvements are most attractive when expected savings, implementation cost, and risk are considered?

2. Source Data and Data Model

Assume the source file is named `pedidos_logistica.csv`, with one row per order. In a production project, confirm the grain of the table first: partial shipments or split deliveries may require a separate shipment-level table rather than a single row per order.

FIELD	DESCRIPTION	VALIDATION
pedido_id	Unique order identifier	Check duplicate IDs and conflicting records
fecha_pedido	Order creation date	Must be a valid date
fecha_prometida	Promised delivery date	Must be valid and not earlier than the order date; apply other business rules
fecha_entrega	Actual delivery date	May be null for an open order
almacen	Distribution center	Trim spaces and standardize labels
transportista	Carrier	Standardize names and aliases
unidades_pedidas	Units requested	Must be greater than zero

FIELD	DESCRIPTION	VALIDATION
unidades_entregadas	Units delivered	Must not be negative; may be null only for open orders; investigate excess quantities
costo_transporte	Transportation cost in soles (S/)	Investigate negative or missing values
estado	Order status	Map source labels to an approved set (e.g. cancelled orders are excluded from OTIF)

3. Data Profiling and Quality Assessment with Python

Start by profiling the raw data before making changes. Preserve an untouched copy of the source and record the number of rows affected by each cleaning rule. Express each issue as a count *and* as a percentage of total rows, since the business question asks for proportions.

```
import pandas as pd
import numpy as np

df = pd.read_csv("pedidos_logistica.csv")
n = len(df)
print("Original dimensions:", df.shape)
print("\nData types:")
print(df.dtypes)

print("\nMissing values (count and % of rows):")
print(pd.DataFrame({"missing": df.isna().sum(),
                    "pct": (df.isna().mean() * 100).round(2)}))

dup_rows = df.duplicated().sum()
dup_ids = df["pedido_id"].duplicated().sum()
print(f"\nExact duplicate rows: {dup_rows} ({dup_rows / n:.2%})")
print(f"Repeated order IDs:   {dup_ids} ({dup_ids / n:.2%})")

print("\nTransportation cost summary:")
print(df["costo_transporte"].describe())
```

This first pass identifies missing values, repeated records, suspicious costs, and unexpected data types. A null actual delivery date may indicate a genuinely open order; it should not automatically be treated as a bad record.

4. Standardization and Cleaning

4.1 Normalize column names, text, dates and numeric fields

```
clean = df.copy()
clean.columns = (
    clean.columns.str.strip()
    .str.lower()
    .str.replace(" ", "_", regex=False)
)

for col in ["pedido_id", "almacen", "transportista", "estado"]:
    clean[col] = clean[col].astype("string").str.strip().str.upper()

# Map aliases to the approved list (example values: replace with the business list).
carrier_map = {"TRANSP. ANDINO": "TRANSPORTES ANDINO", "T. ANDINO": "TRANSPORTES ANDINO"}
status_map = {"ENTREGADO": "ENTREGADO", "ENTREG.": "ENTREGADO", "ANULADO": "CANCELADO"}
clean["transportista"] = clean["transportista"].replace(carrier_map)
clean["estado"] = clean["estado"].replace(status_map)

# Confirm the real source format first; an explicit format avoids misreading dates.
DATE_FORMAT = "%d/%m/%Y"
date_cols = ["fecha_pedido", "fecha_prometida", "fecha_entrega"]
for col in date_cols:
    parsed = pd.to_datetime(clean[col], errors="coerce", format=DATE_FORMAT)
    invalid = parsed.isna() & clean[col].notna()       # present but unparseable
    print(f"{col}: {invalid.sum()} invalid values")
    clean[col] = parsed

numeric_cols = ["unidades_pedidas", "unidades_entregadas", "costo_transporte"]
for col in numeric_cols:
    parsed = pd.to_numeric(clean[col], errors="coerce")
    invalid = parsed.isna() & clean[col].notna()
```

```
    print(f"{col}: {invalid.sum()} non-numeric values")
    clean[col] = parsed
```

4.2 Flag anomalies instead of deleting evidence

```
# Remove only exact duplicate rows; keep an audit of the count first.
exact_duplicates = clean.duplicated().sum()
clean = clean.drop_duplicates()
print("Exact duplicates removed:", exact_duplicates)

# Remaining repeated IDs are conflicting records: flag them, do not pick one silently.
clean["error_id_duplicado"] = clean["pedido_id"].duplicated(keep=False)
duplicate_ids = clean[clean["error_id_duplicado"]].sort_values("pedido_id")
print("Rows involved in duplicate IDs:", len(duplicate_ids))

clean["pendiente"] = clean["fecha_entrega"].isna()

clean["error_cantidad"] = (
    clean["unidades_pedidas"].isna()
    | clean["unidades_pedidas"].le(0)
    | clean["unidades_entregadas"].lt(0)
    | (clean["unidades_entregadas"].isna() & ~clean["pendiente"])
    | (clean["unidades_entregadas"] > clean["unidades_pedidas"])
)

clean["error_costo"] = clean["costo_transporte"].isna() | clean["costo_transporte"].lt(0)

clean["error_fechas"] = (
    clean["fecha_pedido"].isna()
    | clean["fecha_prometida"].isna()
    | (clean["fecha_prometida"] < clean["fecha_pedido"])
    | (
        clean["fecha_entrega"].notna()
        & (clean["fecha_entrega"] < clean["fecha_pedido"])
    )
)

print("Quantity errors:", clean["error_cantidad"].sum())
print("Cost errors (negative or missing):", clean["error_costo"].sum())
print("Date errors:", clean["error_fechas"].sum())
print("Orders without actual delivery date:", clean["pendiente"].sum())
```

Do not automatically keep the first record for every duplicated order ID. Investigate whether repeated IDs are exact copies, corrected versions, or legitimate partial shipments. Resolve them using documented business rules. Keep unresolved cases in an exception table so that data loss is visible.

5. Equivalent Data Quality Checks in SQL Server

SQL can perform scalable checks close to the source. The following examples use SQL Server syntax and assume the source columns have compatible date and numeric types.

5.1 Profile the source

```
SELECT
    COUNT(*)                                              AS total_rows,
    COUNT(DISTINCT pedido_id)                             AS unique_order_ids,
    SUM(CASE WHEN fecha_pedido IS NULL THEN 1 ELSE 0 END)     AS missing_order_dates,
    SUM(CASE WHEN fecha_prometida IS NULL THEN 1 ELSE 0 END)  AS missing_promised_dates,
    SUM(CASE WHEN fecha_entrega IS NULL THEN 1 ELSE 0 END)    AS orders_without_actual_delivery,
    SUM(CASE WHEN unidades_pedidas IS NULL THEN 1 ELSE 0 END) AS missing_units_ordered,
    SUM(CASE WHEN costo_transporte IS NULL THEN 1 ELSE 0 END) AS missing_costs,
    SUM(CASE WHEN costo_transporte < 0 THEN 1 ELSE 0 END)     AS negative_costs
FROM dbo.pedidos_logistica;
```

5.2 Find duplicate order IDs

```
SELECT pedido_id, COUNT(*) AS row_count
FROM dbo.pedidos_logistica
GROUP BY pedido_id
HAVING COUNT(*) > 1;
```

5.3 Identify suspicious records

```
SELECT *
FROM dbo.pedidos_logistica
```

```
WHERE unidades_pedidas IS NULL
   OR unidades_pedidas <= 0
   OR unidades_entregadas < 0
   OR unidades_entregadas > unidades_pedidas
   OR (unidades_entregadas IS NULL AND fecha_entrega IS NOT NULL)
   OR costo_transporte IS NULL
   OR costo_transporte < 0
   OR fecha_pedido IS NULL
   OR fecha_prometida IS NULL
   OR fecha_prometida < fecha_pedido
   OR (fecha_entrega IS NOT NULL AND fecha_entrega < fecha_pedido);
```

Python is well suited to exploration, transformation and statistical analysis. SQL is effective for querying, validating and aggregating large relational tables. A production pipeline can combine both through an ETL/ELT workflow with repeatable checks, logging and traceability.

6. Feature Engineering and KPI Calculation

After unresolved exceptions have been reviewed, create delivery, delay and unit-cost metrics. The example below applies the reporting cutoff described in the methodology note: orders are assigned to the month in which they were **promised**, and only orders whose promised date has passed are evaluated. An open order that is already overdue counts as a failure; an open order that is not yet due is excluded. The final dataset must reflect the documented exception-resolution policy and the business definition of a complete delivery.

```
CUTOFF = pd.Timestamp("2026-06-30")   # example: set to the data extraction / reporting date

valid = clean.loc[
    ~clean["error_cantidad"]
    & ~clean["error_costo"]
    & ~clean["error_fechas"]
    & ~clean["error_id_duplicado"]
    & (clean["estado"] != "CANCELADO")            # cancelled orders are not OTIF-eligible
].copy()

# Eligibility: promised date already passed at the cutoff.
valid = valid.loc[valid["fecha_prometida"] <= CUTOFF].copy()

valid["entrega_completa"] = valid["unidades_entregadas"] >= valid["unidades_pedidas"]
valid["entrega_a_tiempo"] = (
    valid["fecha_entrega"].notna()
    & (valid["fecha_entrega"] <= valid["fecha_prometida"])
)
valid["otif"] = valid["entrega_completa"] & valid["entrega_a_tiempo"]

# Delay: actual delivery date, or the cutoff for overdue open orders.
end_date = valid["fecha_entrega"].fillna(CUTOFF)
valid["dias_retraso"] = (end_date - valid["fecha_prometida"]).dt.days.clip(lower=0)

valid["mes"] = valid["fecha_prometida"].dt.to_period("M")     # period = promised month
valid["costo_por_unidad"] = valid["costo_transporte"] / valid["unidades_pedidas"]

monthly = (
    valid.groupby("mes")
    .agg(
        orders=("pedido_id", "nunique"),
        total_cost=("costo_transporte", "sum"),
        units=("unidades_pedidas", "sum"),
        average_delay_days=("dias_retraso", "mean"),
        otif=("otif", "mean"),
    )
    .reset_index()
)
monthly["cost_per_order"] = monthly["total_cost"] / monthly["orders"]
monthly["otif_pct"] = monthly["otif"] * 100
print(monthly.round(2))
```

Methodology note: For monthly OTIF reporting, assign orders to the period in which they were promised and include only orders whose promised delivery date has passed. Open orders should not be classified as failures before their due date. Because the cleaned set excludes flagged records, report how many orders were excluded so that OTIF and cost per order are not read as covering 100% of the source.

7. Segment Performance by Distribution Center and Carrier

```
def segment_summary(data, key):
    out = (
        data.groupby(key)
        .agg(
            orders=("pedido_id", "nunique"),
            cost=("costo_transporte", "sum"),
            otif=("otif", "mean"),
            average_delay_days=("dias_retraso", "mean"),
        )
        .reset_index()
    )
    out["otif_pct"] = out["otif"] * 100
    out["cost_per_order"] = out["cost"] / out["orders"]
    return out

by_warehouse = segment_summary(valid, "almacen")
by_carrier = segment_summary(valid, "transportista")

print(by_warehouse.sort_values("otif_pct").round(2))
print(by_carrier.sort_values("otif_pct").round(2))
```

These cuts identify where service failures cluster, but they do not prove causality. A carrier may show lower OTIF because it serves longer routes or more difficult delivery locations. Compare route distance, volume, load type, delivery windows and other relevant factors before making a decision. Segments with very few orders should be flagged or grouped, because their percentages are unstable.

8. Quantitative Findings: Illustrative Executive Snapshot

For demonstration, assume the validated analysis produces the following hypothetical results. These are not computed from an actual uploaded dataset.

METRIC	ILLUSTRATIVE RESULT	INTERPRETATION
Orders analyzed	50,000	Comparable orders in the six-month scenario
Logistics cost	S/ 2,400,000	Scenario cost baseline
Cost per order	S/ 48	Total cost divided by order count (2,400,000 / 50,000)
OTIF	88%	Seven percentage points below the 95% target
OTIF orders	44,000	88% of 50,000 orders
Orders not meeting OTIF	6,000	12% of 50,000 orders
OTIF target	95%	Expected service level (47,500 orders)

If OTIF improves from 88% to 95% across 50,000 comparable orders, the number of OTIF orders rises from 44,000 to 47,500: 3,500 additional orders delivered in full and on time. This is a seven-percentage-point improvement, or approximately 7.95% relative growth in OTIF orders (3,500 / 44,000).

Do not treat that service improvement as an automatic cash saving. Financial benefits must be calculated from verified costs such as redelivery, penalties, overtime, returns, expedited freight and customer-service effort.

9. Financial Impact and Scenario Analysis

Suppose operational analysis identifies opportunities to reduce exceptional trips, consolidate dispatches and improve route planning. Use scenarios to estimate potential savings, then validate the assumptions with actual cost data.

SCENARIO	COST REDUCTION	GROSS SIX-MONTH SAVINGS	NET BENEFIT AFTER S/ 30,000 INVESTMENT	ROI
Conservative	3%	S/ 72,000	S/ 42,000	140%
Base case	5%	S/ 120,000	S/ 90,000	300%
Upside	8%	S/ 192,000	S/ 162,000	540%

Formulae: Gross savings = baseline logistics cost x reduction rate. Net benefit = gross savings - project investment. ROI = net benefit / investment x 100. In the base case, a 5% reduction of S/ 2,400,000 equals S/ 120,000 gross savings per six months. After a one-time S/ 30,000 investment, the net benefit is S/ 90,000 and ROI is 300%.

These are scenario assumptions, not a forecast. The table shows a single six-month period; if the savings recur while the investment is one-time, later periods would show a higher return. Confirm the addressable cost base (not every logistics cost can be reduced by these levers), avoid double-counting benefits, and distinguish one-time from recurring savings. The ROI calculation is not meaningful when investment is zero.

10. Power BI Dashboard Design

PAGE	CONTENT	MANAGEMENT QUESTION
1. Executive Summary	OTIF vs. target; total cost; cost per order; late, incomplete and open orders; monthly trend	Are service and cost on track?
2. Operational Performance	Warehouse, carrier and route comparisons; delays; cost per order/unit; incident categories	Where are failures concentrated?
3. Financial Impact	Potential vs. realized savings; incident costs; scenarios; ROI and initiative tracking	Which actions create measurable value?

Example DAX measures (adapt the table and column names to the final Power BI model). The model assumes the table already contains only eligible orders, as defined in Section 6:

```
OTIF Orders =
CALCULATE (
    DISTINCTCOUNT ( Pedidos[pedido_id] ),
    Pedidos[otif] = TRUE ()
)

Total Orders =
DISTINCTCOUNT ( Pedidos[pedido_id] )

OTIF % =
DIVIDE ( [OTIF Orders], [Total Orders], 0 )
```

Use a consistent calendar table (related to the promised date) and a documented rule for order eligibility and the reporting cutoff. Validate dashboard totals against the approved SQL or Python output before publishing.

11. Findings-to-Actions Framework

POTENTIAL FINDING	PROPOSED ACTION	FOLLOW-UP KPI
High cost from exceptional trips	Consolidate dispatches and improve capacity planning	Cost per order; extra trips
Delays concentrated on certain routes	Review route design and delivery windows	OTIF by route
Repeated carrier service failures	Review root causes and service-level agreements	OTIF by carrier
Picking or quantity discrepancies	Strengthen picking and dispatch verification	Complete-delivery rate
Recurring source-data errors	Automate validation and exception reporting	Data error rate

Prioritize initiatives by expected benefit, implementation cost, time to impact and confidence in the evidence. Where possible, compare performance before and after implementation and account for changes in volume, mix and seasonality.

12. Deliverables and Project Completion Criteria

- Clean dataset with a documented audit trail and exception log.
- Reusable Python notebook or script for profiling, cleaning, validation and KPI generation.
- SQL queries for data-quality checks and analytical aggregations.

- Power BI dashboard for operational and financial monitoring.
- Executive report describing findings, root-cause hypotheses, recommendations and quantified opportunities.
- Measurement plan with owners, targets, reporting frequency and savings-validation rules.

Appendix: Review Log (corrections made to the original)

The arithmetic in Sections 8 and 9 was re-checked and is correct (cost per order, OTIF counts, 7.95% relative growth, savings, net benefit and ROI). The corrections below concern code logic, consistency and completeness.

#	SECTION	ISSUE IN THE ORIGINAL	CORRECTION
1	6	Code treated open orders as OTIF failures, contradicting the methodology note (open, not-yet-due orders must not fail).	Added reporting CUTOFF; only orders with promised date on or before the cutoff are evaluated.
2	6	Month was based on the order date, while the note says to use the promised date.	mes now uses fecha_prometida.
3	4.2 / 6	Conflicting duplicate order IDs stayed in the data; nunique counted once but costs were summed on every row.	Added error_id_duplicado flag and excluded those rows from the valid set (kept visible in an exception table).
4	4.2 / 5.3	Missing costs and missing ordered units were not flagged, although Section 2 says to investigate them.	error_costo and error_cantidad now include nulls; SQL 5.1 and 5.3 updated.
5	4.2 / 5.3	No check that the promised date is not earlier than the order date.	Added to error_fechas and to the SQL query.
6	4.2	Null delivered units on a delivered order were not detected.	Flagged when units delivered is null and the order has an actual delivery date.
7	4.1	dayfirst=True can misread ISO dates; invalid dates and non-numeric values were silently coerced to null.	Explicit date format; the count of unparseable values is printed per column.
8	4.1	Aliases and status labels were only upper-cased; Section 2 requires mapping to an approved set.	Added example alias and status maps.
9	3	Question 1 asks for proportions, but profiling printed only counts.	Counts are now shown with percentages of total rows.
10	6	Cancelled orders were not addressed in the OTIF denominator.	Cancelled orders excluded; delay for overdue open orders measured up to the cutoff.
11	7	Carrier table lacked cost per order; code was duplicated.	Single reusable function; both tables include cost per order; low-volume caution added.
12	8-10	Snapshot omitted OTIF orders count; ROI table ignored recurrence; DAX had no eligibility note.	Added OTIF orders row, recurrence caveat and DAX eligibility note.
13	Cover / formatting	Text layout issues from the original: code blocks split by page headers, 'endto-end' hyphenation.	Reflowed layout; code blocks kept intact and legible.